LESSON PLAN · SCRATCH (GR 3–5) BEGINNER: SCRATCH GAME MAKERS

# Lesson 11: Messages, Game Over and Debugging

Module 4 | 45 minutes | Broadcasts, game states and debugging | Grades 3–5 · ages 8–10

**Objective:** Use broadcast messages to coordinate sprites, add win and game-over endings with a timer, and debug programs step by step.

**Success criteria**

- I can send and receive messages between sprites.
- I can add a win or game-over ending.
- I can use a debugging routine to fix a bug.

## Key vocabulary

- **broadcast**: Send a message to all sprites.
- **receive**: Start a script when a message arrives.
- **timer**: Counts seconds since it was reset.
- **debug**: Find and fix mistakes in code.

## You will need

- Projector with Kitabash (lesson video, Code Lab)
- Devices: 1 per learner or pair
- Printed worksheet (optional)

## Timed lesson plan (45 min)

| Time | Stage | What happens |
|---|---|---|
| 5 min | **Warm-up (unplugged)** | School-bell messages. |
| 5 min | **Watch** | Lesson video; pause on the debugging steps. |
| 8 min | **Build along** | Win and game-over screens. |
| 14 min | **Perform** | Fix the square, you-win Code Lab, order and match activities. |
| 9 min | **Probe** | Fix the game over, fix the maze runner, debugging strategy, bug story. |
| 4 min | **Wrap-up** | Exit ticket; plan capstone teams if needed. |

## Unplugged warm-up

School bell (5 min): assign actions to messages (“lunch” = stand, “home time” = wave). Ring a bell and say a message; only listeners for that message act. Then say a wrong message name — nobody moves: a bug!

## Teaching the idea

Broadcasts decouple sprites: one sprite announces an event; any number of receivers react. Game states (playing, win, game over) are often driven by broadcasts plus “wait until” on a variable or the timer. Debugging is taught as a routine: expected vs actual, read aloud, check hats/numbers/names, change one thing.

**Analogy:** A broadcast is the school bell: one ring, and everyone who is listening for it knows what to do.

**Live demo:** Show the win sequence with two sprites. Then plant a message-name typo and model the debugging routine aloud.

## Questions to ask

| Ask | Listen for |
|---|---|
| Who sends and who receives? | Name the broadcaster and the receivers. |
| Why did nothing happen? | Message names don’t match / no receiver / no hat. |
| What’s the first thing you check when debugging? | Expected vs actual; then hats. |

## Differentiation

**Support**
Use only the you-win lab with message names pre-typed; students add the show and say blocks.

**Stretch**
Add a three-state game: title screen (press space to start), playing, game over — all with broadcasts.

## Common misconceptions

| Misconception | How to fix it |
|---|---|
| A broadcast goes to one sprite | It goes to every sprite (and the stage); only those with a matching hat react. |
| Message names can be close enough | They must match exactly. |
| stop all only stops the current sprite | It stops every script in the project. |

## Assessment

Use the Code Lab results (auto-checked), the quick checks and the Probe answers. Written answers arrive in your review queue.

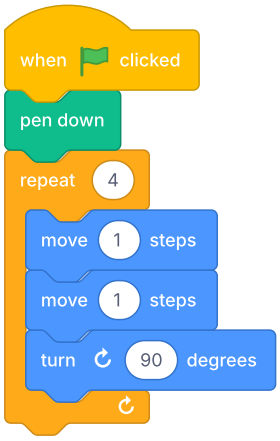
**Exit ticket**
Write the 4 steps you will use next time you find a bug.

**Homework**
Sketch your capstone game idea: what is the goal, how do you win, how do you lose?

# Answer key — Kitabash activities

## 1. Fix the square (perform · block code)

**One solution:**

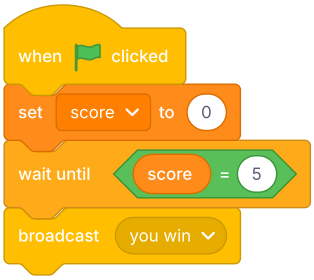


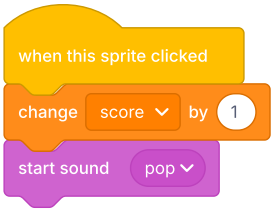
## 2. Code Lab: you win! (perform · code lab)

**Goals checked:** Star broadcasts “you win” · Trophy is hidden at the start · Trophy says “You win!” · Trophy appears

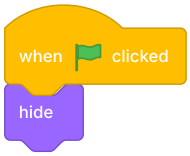
**One solution:**

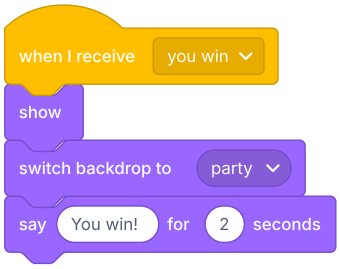
### Star




### Trophy




## 3. Follow the message (perform · ordering)

**Order:** The player catches the 5th apple → The score becomes 5 → The Star’s “wait until” finishes → It broadcasts “you win” → The Trophy receives it and shows itself

## 4. Match each block to what it does (perform · matching)

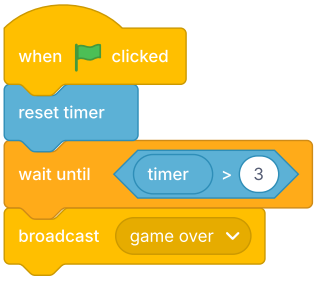
broadcast (game over) → Sends a message to every sprite · when I receive (game over) → Starts a script when that message arrives · stop all → Stops every script in the project · reset timer → Sets the timer back to 0

## 5. Code Lab: fix the game over (probe · code lab)

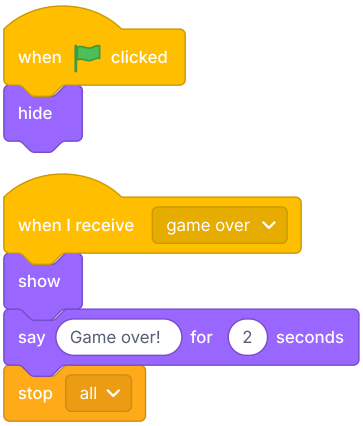
**Goals checked:** The Banner says “Game over!” · The Banner appears

**One solution:**

### Star

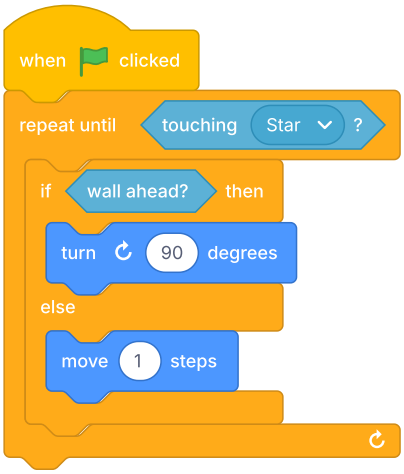



### Banner




## 6. Fix the maze runner (probe · block code)

**One solution:**




## 7. Debugging strategy (probe · multiple choice)

**Answer:** That the script has a “when green flag clicked” hat on top

**8. Explain your thinking** (probe · free response)

**Look for:** expected, happened, fixed. **Example:** I expected my apple to fall again after reaching the bottom, but it stayed stuck. My if block was outside the forever loop, so I moved it inside and it fixed it.

## Answer key — worksheet

1. **Who reacts?:** Both scripts with the exact name “level up”.
2. **Find the bug:** The message names differ (“game over” vs “Game Over”). Make them exactly the same.
3. **My debugging routine:** Expected vs actual; read aloud; check hats; check numbers/names; change one thing and test.
4. **Plan an ending:** A clear win/lose condition with broadcast + when I receive.

Kitabash original curriculum. Scratch is a project of the Scratch Foundation in collaboration with the MIT Media Lab; this course is not affiliated with or endorsed by them.